Tecnicatura Superior en Desarrollo de Software

Módulo 08 — Infrastructure: Persistencia, `DbContext` y Repositorios Concretos

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

¿Qué es la capa Infrastructure?

Infrastructure	Capa de acceso a datos, persistencia y configuración del host	Capa: Periférica (externa a Clean Architecture)

La capa **Infrastructure** es donde viven los detalles técnicos de *cómo* el sistema se comunica con el mundo exterior: acceso a base de datos, configuración de la ORM (Entity Framework Core), host de la aplicación (ventana WPF, servidor web), integración con servicios externos. Es la capa más baja y más específica de tecnología de Clean Architecture.

Las capas internas (Domain, Application, InterfaceAdapter) no saben nada de Infrastructure — ni saben si la base de datos es SQL Server, PostgreSQL o un archivo en memoria. Infrastructure implementa las interfaces definidas por Application (repositorios, puertos de salida, adaptadores) y las saca adelante con la tecnología que elija.

Componentes principales:

- **DbContext** — mapeo objeto-relacional (ORM)
- **Repositorios concretos** — implementan las interfaces de Application
- **Migraciones** — versionado de esquema de base de datos
- **Host / Composition Root** — inyección de dependencias global
- **Configuración** — connection strings, logging, etc.

Hasta ahora, en los módulos anteriores, los repositorios y los puertos de salida eran solo *interfaces* — contratos que las capas internas esperaban que alguien implementara. La capa Infrastructure es exactamente ese alguien.

Analogía del mundo real

Imaginá que los módulos anteriores diseñaron una **estación de correos** (tu aplicación). Cada empleado de la estación sabe qué hacer (recibe paquetes, los valida, los procesa) y habla con sus compañeros a través de turnos (InputPort, OutputPort). Pero nadie sabe aún *dónde* se guardan los paquetes, *con quién* contratan para el envío, ni en *qué edificio* funciona la estación.

Infrastructure responde eso: es el edificio físico, el almacén de la esquina (base de datos), los contratos con empresas de envío (servicios externos). Sin Infrastructure, la estación de correos es un proceso abstracto que nunca toca el mundo real.

Cuando llega un paquete por InputPort, la aplicación no dice "voy a guardarlo en SQL Server" — simplemente llama al repositorio (una interfaz), y es Infrastructure quien decide que eso significa hacer un INSERT en una tabla de SQL Server.

Características esenciales

La capa Infrastructure tiene responsabilidades muy concretas:

1. Implementación de repositorios

Application define `IProductRepository` (interfaz). Infrastructure crea `ProductRepository` (clase concreta que accede a SQL Server). El repositorio convierte llamadas de métodos en comandos de base de datos (SELECT, INSERT, UPDATE, DELETE).

2. DbContext y mapeo objeto-relacional

Entity Framework Core traduce entre tu modelo de dominio (clases C#, Value Objects, Entidades) y las tablas de la base de datos. El `DbContext` es el corazón: contiene el modelo de la BD completo (tablas, relaciones, restricciones) y es por dónde pasa *todo* acceso a datos.

3. Configuración Fluent API

Entity Framework permite dos formas de describir cómo mapear: anotaciones (atributos `[Column]`, `[Key]`) o `OnModelCreating` — código C# que describe el esquema. Clean Architecture prefiere `OnModelCreating` para evitar contaminar el dominio con metadatos de persistencia.

> **Tema avanzado:** Anexo 08a — Configuración Fluent API cubre en profundidad cómo mapear Value Objects, relaciones, índices, restricciones y Data Seeding. Consultalo para dominios complejos.

4. Migraciones

Cuando la estructura de la BD cambia (agregar tabla, columna, índice), la migración captura ese cambio como un script SQL versionado que se puede ejecutar, revertir o auditar. No edites la BD a mano — déjale eso a las migraciones.

5. Composition Root y Host

Infrastructure es también donde se arma la cadena completa de inyección de dependencias. El `DependencyContainer` (ya visto en InterfaceAdapter) vive en la capa de presentación, pero Infrastructure configura la base: registra los repositorios concretos, el DbContext, las cadenas de conexión, y el host de la aplicación (la ventana WPF o el servidor).

¿Por qué esta estructura?

Invertir las dependencias. Las capas internas dicen "necesito guardar datos", no "necesito SQL Server". Cuando Infrastructure implementa esa interfaz, las capas internas siguen funcionando igual — solo cambió el detalles de *cómo* se guarda, no *qué* se hace.

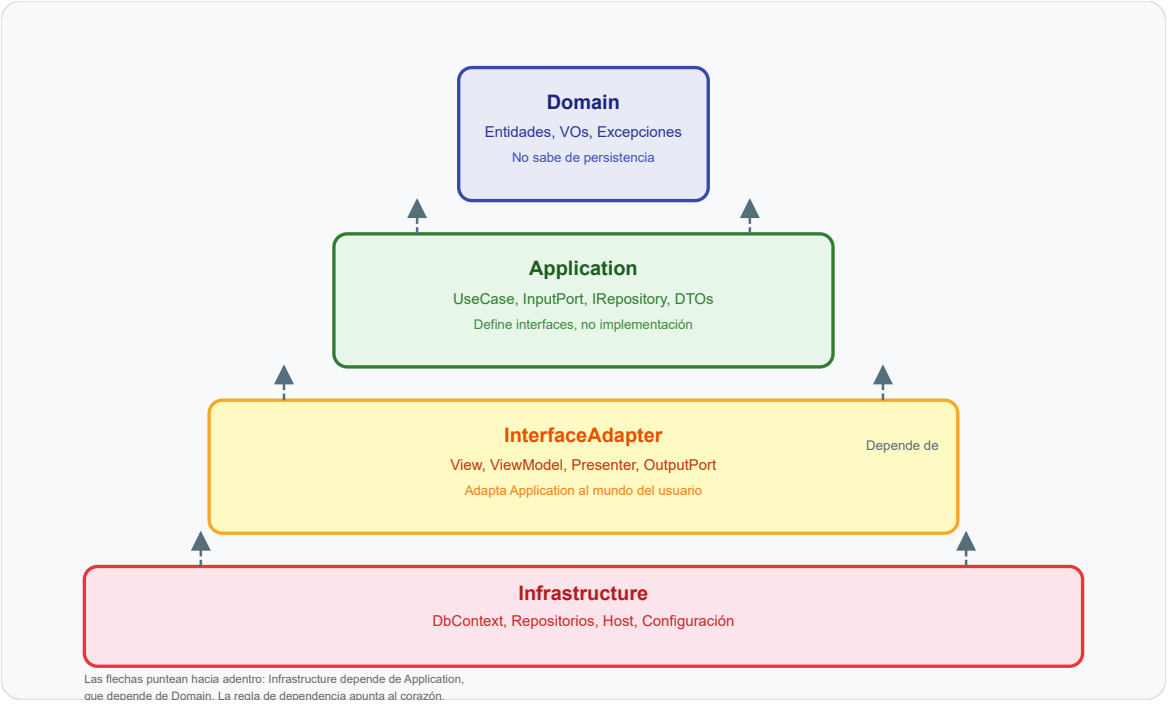
Testear sin base de datos. Application tiene tests que usan un `IProductRepository` falso (en memoria, sin tocar la BD). Cuando llega a InterfaceAdapter o Infrastructure, los tests usan uno real. La arquitectura permite ambos sin duplicar código de aplicación.

Cambiar de BD sin rediseñar. Hoy es SQL Server. Mañana es PostgreSQL. La aplicación no cambia — solo escribís un nuevo `ProductRepository` que hablé con PostgreSQL. Application nunca supo que SQL Server existía.

Separación clara de responsabilidades. Domain no sabe de persistencia. Application no sabe de SQL. InterfaceAdapter no sabe de inyección de dependencias a nivel de BD. Cada capa sabe lo que le toca — Infrastructure hace el resto.

¿Cómo encaja en la arquitectura?

Mirá de nuevo las capas de Clean Architecture:



Las capas internas no conocen Infrastructure. Application no `using` nada de Infrastructure — solo define interfaces. Es Infrastructure quien busca esas interfaces y las implementa.

Dirección de flujo de datos: Persona → InterfaceAdapter (ViewModel) → Application (UseCase) → Domain (Entidades, Excepciones) → Application (OutputPort) → InterfaceAdapter (Presenter, View) → Persona. En el medio de eso, Application llama al repositorio (interfaz) — Infrastructure está al otro extremo de esa línea, esperando.

Referencias del proyecto: de dónde sale el código que usás

En .NET nada se "importa" por arte de magia. Un proyecto solo puede usar el código que **declaró de manera explícita** en su archivo `.csproj`. Si escribís `using Microsoft.EntityFrameworkCore;` en un proyecto que no declaró esa librería, el compilador no la encuentra y falla. Antes de configurar Entity Framework Core hay que entender cómo se le da a un proyecto acceso a código que no escribió él.

Dos tipos de referencia

Una **referencia a proyecto** (`ProjectReference`) le da acceso al código de otro proyecto de *la misma solución*: así Infrastructure puede usar las entidades de Domain. Un **paquete NuGet** (`PackageReference`) le da acceso a una librería ya compilada que publicó otra persona u organización; NuGet es el almacén público de esas librerías. Un paquete tiene un *nombre* y una *versión*; al agregarlo, NuGet lo descarga una sola vez a una carpeta de tu máquina (no a tu repositorio) y el compilador lo usa desde ahí. A esa descarga se la llama *restauración* y ocurre sola la primera vez que compilás.

Así se ve en el .csproj de Infrastructure:
La referencia a proyecto y los paquetes son líneas del archivo. No hay nada más: todo lo que el proyecto puede usar está escrito ahí.

```
<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>
    <!-- referencia a otro proyecto de la misma solución -->
    <ProjectReference Include="..\Domain\LibraryApp.Domain.csproj" />
  </ItemGroup>

  <ItemGroup>
    <!-- referencias a paquetes NuGet: librerías de terceros, con nombre y versió
n -->
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version
="9.0.20" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="9.
0.20">
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
  </PropertyGroup>

</Project>
```

Qué paquetes necesita Infrastructure, y para qué

- `Microsoft.EntityFrameworkCore.SqlServer` — Entity Framework Core más el *proveedor* de SQL Server, la pieza que traduce tus consultas en C# al SQL de ese motor. Es lo que habilita `DbContext`, `DbSet` y `UseSqlServer()`. Si mañana cambiás de motor, cambiás este paquete (y la línea de configuración), no el negocio.
- `Microsoft.EntityFrameworkCore.Design` — lo que necesitan las herramientas de migraciones para analizar el proyecto. No agrega nada a tu código; por eso se marca `PrivateAssets` con valor `all`: sirve para trabajar en este proyecto pero no se propaga a quienes lo referencien.
- `dotnet-ef` — no es un paquete del proyecto sino una *herramienta* de línea de comandos que se instala una sola vez en la máquina (`dotnet tool install --global dotnet-ef`) y ejecuta los comandos `dotnet ef migrations` y `dotnet ef database`.

Las referencias materializan la regla de dependencias

La regla de Clean Architecture ("las dependencias apuntan hacia adentro") no es solo una idea: se cumple o se rompe en los `.csproj`. Domain no referencia a nadie. Application referencia a Domain. InterfaceAdapter referencia a Application. Infrastructure referencia a Domain y a Application según los contratos que implemente, y **es la única que referencia a Entity Framework Core**. Si un proyecto interno tuviera un `PackageReference` a EF Core, ya habría quebrado la arquitectura antes de escribir una sola línea de lógica.

Cómo se agrega un paquete

Hay dos caminos equivalentes que producen exactamente el mismo cambio en el `.csproj`. Por línea de comandos, desde la raíz de la solución:

```
dotnet add src/Infrastructure/LibraryApp.Infrastructure.csproj package Microsoft.EntityFrameworkCore.SqlServer --version 9.0.20
dotnet add src/Infrastructure/LibraryApp.Infrastructure.csproj package Microsoft.EntityFrameworkCore.Design --version 9.0.20
```

Desde Visual Studio: clic derecho sobre el *proyecto* (no la solución), **Administrar paquetes NuGet…**, pestaña **Examinar**, buscar el paquete, elegir la versión e **Instalar**. La actividad práctica de este módulo recorre los dos caminos paso a paso.

> **La versión importa:** cada versión mayor de EF Core exige la versión mayor equivalente de .NET. En un proyecto `net9.0` corresponde EF Core 9.x; instalar la 10.x produce un error de compatibilidad (`NU1202`). Fijá siempre la versión exacta para que todo el equipo compile lo mismo.

ERROR COMUN

Agregar Entity Framework Core a la capa de dominio:

```
<!-- LibraryApp.Domain.csproj — MAL -->
<ItemGroup>
  <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.20" />
</ItemGroup>
```

El negocio queda atado a una tecnología de persistencia: ya no podés cambiarla ni probar el dominio sin ella.

BUENA PRACTICA

El paquete va solo en Infrastructure:

```
<!-- LibraryApp.Domain.csproj — BIEN: sin paquetes de terceros -->
<!-- LibraryApp.Infrastructure.csproj — BIEN: acá sí va EF Core -->
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.20" />
```

Entity Framework Core: DbContext y Configuración

¿Qué es DbContext?

`DbContext` es el puente entre tu modelo de dominio (clases C#) y la base de datos. Contiene:

- **DbSet<T>** — colecciones que mapean a tablas
- **OnModelCreating()** — configuración Fluent API del esquema

- **SaveChanges()** — persiste cambios a la BD
- **Connection string** — dónde vive la BD

Configuración Fluent API

En lugar de poner `[Table]`, `[Column]`, `[Required]` en tus clases de dominio (contaminando el código de negocio con metadatos de persistencia), usás `OnModelCreating()` para describir el esquema en C# puro:

```
public class LibraryDbContext : DbContext
{
    public DbSet<Book> Books { get; set; }
    public DbSet<Author> Authors { get; set; }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        // Configurar la tabla de libros
        builder.Entity<Book>().ToTable("Books");
        builder.Entity<Book>().HasKey(b => b.Id);
        builder.Entity<Book>().Property(b => b.Title)
            .HasMaxLength(200).IsRequired();
    }
}
```

Fluent API te permite encadenar métodos (`.HasKey()`, `.HasMaxLength()`, `.IsRequired()`) para construir un esquema completamente tipado, sin magia ni atributos esparcidos por el dominio.

> **Qué le exige EF Core a tus entidades:** para reconstruir un objeto desde la base necesita un constructor que pueda llamar. Si la entidad solo tiene constructores con Value Objects como parámetros (no se pueden enlazar), hace falta además un constructor `private` sin parámetros. Y las propiedades de solo lectura (`{ get; }`) **no se mapean por convención**: hay que declararlas con `Property()` o se pierden en silencio al guardar. Los dos casos, con ejemplos, están en el anexo 08a.

Una clase de configuración por entidad

Escribir toda la configuración dentro de `OnModelCreating()` funciona mientras hay una o dos entidades. Con diez o veinte, ese método se vuelve una pared de código. EF Core ofrece la interfaz `IEntityTypeConfiguration<T>` para

repartirla: **una clase por entidad**, en una carpeta propia (por ejemplo `Data/Configurations/`), y el `DbContext` solo las enumera.

```
public sealed class BookConfiguration : IEntityTypeConfiguration<Book>
{
    public void Configure(EntityTypeBuilder<Book> book)
    {
        book.ToTable("Books");
        book.HasKey(b => b.Id);
        book.Property(b => b.Title).HasMaxLength(200).IsRequired();
    }
}
```

El `DbContext` queda reducido a declarar las tablas y aplicar cada configuración:

```
public sealed class LibraryDbContext : DbContext
{
    public LibraryDbContext(DbContextOptions<LibraryDbContext> options)
        : base(options)
    {
    }

    public DbSet<Book> Books => Set<Book>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // el DbContext solo enumera las configuraciones
        modelBuilder.ApplyConfiguration(new BookConfiguration());
    }
}
```

Qué se gana:
El `DbContext` no crece con cada entidad; cada mapeo está en un único archivo corto; y las configuraciones describen el modelo, no el proveedor, así que sobreviven a un cambio de motor de base de datos sin tocarlas. Es un movimiento de código: el esquema resultante no cambia.

> **Profundidad completa:** Este módulo muestra lo básico de Fluent API. Para casos avanzados —mapear Value Objects con `OwnsOne()` / `OwnsMany()`, relaciones many-to-many, índices compuestos, Data Seeding, y patrones de configuración— consultá Anexo 08a.

El patrón Repository

Application define `IBookRepository`. Infrastructure implementa `BookRepository`. El repositorio es el adaptador que:

1. Recibe datos del mundo de Application (objetos de dominio, valores escalares)
2. Traduce a comandos SQL (o consultas LINQ a través de EF)
3. Devuelve objetos de dominio (o listas, o `null` si no encuentra)

El repositorio **no expone SQL** — expone métodos con nombres de negocio: `FindById()`, `FindByEmail()`, `AddAsync()`, `UpdateAsync()`. Quien lo llama no sabe si adentro se ejecuta SQL Server o una lista en memoria.

Ejemplo: BookRepository

```
1  public sealed class BookRepository : IBookRepository
2  {
3      private readonly LibraryDbContext _db;
4
5      public BookRepository(LibraryDbContext db) => _db = db;
6
7      public async Task<Book?> FindByIdAsync(Guid id)
8          => await _db.Books.FirstOrDefaultAsync(b => b.Id == id);
9
10     public async Task AddAsync(Book book)
11     {
12         await _db.Books.AddAsync(book);
13         await _db.SaveChangesAsync();
14     }
15 }
```

El repositorio inyecta el `DbContext` (el puente a la BD). Cada método mapea una operación de negocio (agregar un libro, buscar por ID) a una operación EF (`.AddAsync()`, `.FirstOrDefaultAsync()`). La persistencia es responsabilidad del repositorio, no de quien lo llama.

Errores comunes

ERROR COMUN

Exponer SQL directo en el repositorio o Domain:

```
// MAL — el UseCase ve SQL
var books = await db.Books
    .FromSqlRaw("SELECT * FROM Books WHERE Price > {0}", price).ToListAsync
();
```

Application queda acoplada a SQL. Si cambias la BD, el UseCase se rompe. Infrastructure debería encapsular esto en un método con nombre de negocio.

BUENA PRACTICA

Encapsular en un método del repositorio:

```
// BIEN — el UseCase solo sabe que existen libros caros
public async Task<List<Book>> FindExpensiveBooksAsync(decimal minPrice)
    => await _db.Books.Where(b => b.Price > minPrice).ToListAsync();
```

ERROR COMUN

Olvidar `SaveChangesAsync()` después de cambios:

```
await _db.Books.AddAsync(book);
// Falta: await _db.SaveChangesAsync()
```

El libro se agrega en memoria pero **nunca se persiste** a la BD. Los cambios se pierden cuando se destruye el DbContext.

BUENA PRACTICA

Siempre persisitir explícitamente:

```
await _db.Books.AddAsync(book);
await _db.SaveChangesAsync(); // Ahora sí persiste
```

ERROR COMUN

Violar la dirección de dependencias: Application usa Infrastructure:

```
using LibraryApp.Infrastructure; // MAL en Application
public class AddBookUseCase
{
    private readonly BookRepository _repo; // Acoplado a concreto
```

Application depende de una clase concreta de Infrastructure. Si cambias el repositorio, Application se rompe.

BUENA PRACTICA

Inyectar la interfaz, que Infrastructure implementa:

```
public class AddBookUseCase
{
    private readonly IBookRepository _repo; // Depende de interfaz
    public AddBookUseCase(IBookRepository repo) => _repo = repo;
```

El Host de Escritorio y Composition Root

Hasta ahora, la aplicación vive en abstracciones: InputPort, UseCase, OutputPort. Pero en algún lugar tiene que alojarse en el mundo real. Para una aplicación de escritorio, ese lugar es **WPF (Windows Presentation Foundation)** — la ventana que el usuario ve.

El *host* es ese programa principal (la ventana .xaml + su code-behind). El *composition root* (o DI container) es donde se arman todas las dependencias: se registra el DbContext, el repositorio, el UseCase, el Presenter, el ViewModel, y se wiring todo para que cuando el usuario haga clic en "Guardar", la cadena completa funcione.

Infrastructure es responsable de esa configuración: el archivo `Program.cs` o `DependencyInjectionExtensions.cs` donde se dicen las reglas — "siempre que alguien pida un `IBookRepository`, dame un `BookRepository`; siempre que pida un `LibraryDbContext`, dame un contexto nuevo por scope", etc.

Qué NO hacer

QUE NO HACER

Dejar la lógica de negocio en el repositorio:

```
public async Task AddBookIfNotExpensive(Book b)
{
    if (b.Price < 100) // Lógica de negocio, no debe estar acá
        await AddAsync(b);
}
```

La regla "no agregar libros caros" es lógica de negocio. Pertenece al UseCase (Application), no al repositorio (Infrastructure).

BUENA PRACTICA

La lógica queda en Application, infraestructura solo persiste:

```
public class AddBookUseCase
{
    public async Task ExecuteAsync(AddBookRequest req)
    {
        if (req.Price >= 100)
            throw new ExpensiveBookException();
        var book = Book.Create(req.Title, req.Price);
        await _repo.AddAsync(book); // Repository solo persiste
    }
}
```

Resumen

Infrastructure es la capa que toca el mundo real.

Implementa las interfaces de Application (repositorios, servicios) usando la tecnología que elijas (SQL Server, PostgreSQL, MongoDB, servicios web, etc.). El DbContext es el corazón: mapea objetos de dominio a tablas. El repositorio encapsula queries y comandos. El host WPF aloja todo. La dirección de dependencias apunta hacia adentro: Infrastructure depende de Application, que depende de Domain. Domain no sabe de persistencia.

Resultado: una aplicación que puede cambiar de BD, ser testeada sin BD, o reemplazar la ventana WPF por un servidor web, sin tocar una sola línea de lógica de negocio.

Módulo 08 — Infrastructure: Persistencia, DbContext y Repositorios Concretos | Programación II | IES N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026